Credit Default Prediction Tool Final Report

Chris Tahan & Mahamadou Cissé

Dataset: Default of Credit Card Clients (UCI id 350, CC BY 4.0)

Repository: c1vxs/credit-default

Summary:
We built a tool that tells a bank's credit risk/collections team each month which card holders are most likely to miss next month's payment and therefore who needs to be acted on early before they actually default. The final model is a k-nearest-neighbours classifier (k = 201) inside a scikit-learn pipeline, with a decision threshold of 0.19 chosen on out-of-fold predictions to catch 60 % of defaulters. On the test set, used once, it catches 811 of 1,327 defaulters (recall 0.611) with precision 0.436, twice the 22 % base rate. Test scores match the cross-validation scores, so the model generalises.

1. Problem framing

The tool supports one monthly decision: which credit card clients the bank's credit risk team should contact before the next payment is due. Early actions include a reminder call, a payment plan or a review of the credit limit. The tool ranks clients by risk and a person makes the final decision.

- Target: default payment next month, renamed default (1 = the client defaults in October 2005, 0 = pays).
- Task: binary classification on imbalanced classes (22.1 % positives).
- Question answered: will this client default on next month's payment?

The two mistakes do not cost the same:

Mistake	What happens	Cost
False negative (missed defaulter)	No action is taken, the client defaults	The unpaid balance is lost; the client's debt grows
False positive (false alarm)	A client who would have paid is contacted	One needless call, an annoyed client, possibly an unfair limit review

A missed defaulter costs more than a needless call, so we favor the recall: the share of real defaulters the tool catches. The call volume still has limits, so we fix a recall goal (60 %) instead of flagging everyone. Accuracy is not used: a model that predicts "no default" for everybody already reaches 77.9 % while catching no defaulter.

2. Data: exploration, cleaning and leakage

The data holds 30,000 clients of a Taiwanese bank (April–September 2005), 23 features and an ID column. There are no missing values, and 6,636 clients (22.1 %) default.

Group	Columns	Meaning
Credit	LIMIT_BAL	Credit limit (NT dollars)
Profile	SEX, EDUCATION, MARRIAGE, AGE	Coded categories, age in years
Repayment status	PAY_0, PAY_2 … PAY_6	Months of delay, September back to April
Amounts	BILL_AMT1–6, PAY_AMT1–6	Amount billed and amount paid each month

Special codes: We checked every coded column with value_counts against the UCI documentation.

- EDUCATION has codes 0, 5 and 6, which are not documented. They are merged into 4 (“others”). MARRIAGE code 0 is merged into 3 (“others”). No row is lost; a test checks it.
- PAY_ uses −2 and 0, which are not documented either. Their default rates (13 % each) are as low as −1 (“paid on time”, 17 %), so all three mean “not late”. The values stay numeric because their order (more delay = more risk) is meaningful.
- Negative BILL_AMT values are overpayments (a credit on the card). They are real, so we keep them.

Leakage: The target is the October payment. Every feature describes April to September, so all of them are known before the event and none needs removing. ID is a row number with no meaning: the ColumnTransformer selects only the 23 features, so ID never reaches the model (a notebook check asserts it).
Split: We split once, before any modelling: 80 % train (24,000 clients) and 20 % test (6,000), stratified on the target with a fixed seed (42). Both sets keep the 22.1 % default rate. The test set was then left aside until the final evaluation (section 6).

3. Method: pipeline and validation protocol

Every preparation step lives inside one scikit-learn Pipeline, so cross-validation refits it on each training fold and nothing leaks from the validation fold.

1. **ColumnTransformer**: StandardScaler on the 20 numeric columns (LIMIT_BAL, AGE, the six PAY_, BILL_AMT and PAY_AMT columns). OneHotEncoder(handle_unknown="ignore") on SEX, EDUCATION and MARRIAGE, because their codes are labels, not quantities. All other columns (ID) are dropped.
2. **Classifier**: the model being compared.

Scaling matters for two reasons: k-NN measures distances, so without scaling LIMIT_BAL (up to 1,000,000) would drown AGE (21–79). Regularised linear models also need features on the same scale for a fair penalty. Because the scaler sits inside the pipeline, its mean and standard deviation are learned on the training folds only; a pytest test checks that the fitted mean_ equals the training mean. We added no engineered features and needed no imputation (no missing values).
Protocol (Labs 1 and 2). All choices are made on the 24,000 training clients with the same StratifiedKFold (5 folds, shuffled, seed 42), so every model sees exactly the same folds. Each model is tuned with GridSearchCV, ranked by average precision (section 5 explains why).

Model	How it decides	Hyperparameter tuned
DummyClassifier (most frequent)	Always predicts “no default”	— (baseline)
Logistic regression	Weighted sum of the features → sigmoid → probability	C, inverse of the regularisation strength
k-NN	Share of defaulters among the k most similar clients	n_neighbors k: 15, 25, 51, 75, 101, 151, 201, 301

Model	How it decides	Hyperparameter tuned
Linear SVM (LinearSVC)	Separating line with the widest margin	C

In the first run the best k was 51, the largest value of the grid. An optimum on the edge of a grid may not be the real one, so we extended the grid to 301 and re-ran the same cross-validation. The best k became 201, now inside the grid. This decision came from the CV results only.

4. Results: baseline and model comparison

All three models more than double the baseline's average precision. k-NN ranks first, but only by 0.010, which is less than the ±0.013 its score varies from one fold to another. So k-NN and logistic regression perform about the same.

Model (best setting)	CV average precision (mean ± std)	CV ROC-AUC	Recall at 0.5	Precision at 0.5
k-NN (k = 201)	**0.517 ± 0.014**	**0.760**	0.271	0.662
Logistic regression (C = 0.1)	0.507 ± 0.013	0.727	0.244	0.711
Linear SVM (C = 1)	0.504 ± 0.013	0.724	0.176	0.724
Dummy (most frequent)	0.221	0.500	0.000	—

5-fold stratified cross-validation on the 24,000 training clients. The dummy's average precision equals the default rate, the score of a model with no skill.

Choice of the final model: We fixed the rule before seeing the scores: the best CV average precision among the models that output probabilities. The code applies it, so the choice is not made by hand.

- k-NN wins with 0.517. It also has the best ROC-AUC and recall; logistic regression has the best precision at 0.5.
- The gap is small. 0.010 is about one standard deviation (± 0.013–0.014), so k-NN and logistic regression are statistically tied. We say so rather than oversell the win.
- The linear SVM cannot be the final model. LinearSVC returns a score, not a probability (predict_proba), and our threshold step needs probabilities.
- Logistic regression is kept for interpretation (section 7), since k-NN has no coefficients.

At the default threshold of 0.5, every model catches at most 27 % of defaulters. This is far too few for the bank, which is why the threshold is chosen separately.

5. Metric and decision threshold

We use one metric to choose the model and another to choose the decision: average precision ranks the models, then a 60 % recall goal sets the threshold at 0.19.

Average precision to choose the model. It summarises the precision–recall curve over all thresholds, so it does not depend on a threshold we had not chosen yet. It focuses on the rare class we care about: a model with no skill scores 0.22 (the default rate), whereas **ROC-AUC gives it 0.5 and accuracy 0.78**. Under imbalance, this makes it the most demanding of the three.

A recall goal to choose the decision. Section 1 says a missed defaulter costs more than a false alarm, so the goal is to catch at least 60 % of defaulters. Among the thresholds that reach it, we keep the highest one, because it flags the fewest clients and so gives the best precision:

Rule: threshold = the highest t in {0.01, 0.02, …, 0.99} such that out-of-fold recall(t) ≥ 0.60.
Out-of-fold, never the test set. The probabilities come from cross_val_predict (same 5 folds) on the training set: each client is scored by a model that never saw them. In-sample probabilities would be too optimistic, especially for k-NN, where a training client is its own nearest neighbour. Using the test set here would make the final score optimistic.
Why 60 %. It is a business choice; the table shows what each extra 10 points of recall costs in calls (out-of-fold, training set).

Recall goal	Share of clients flagged	Precision
60 % (chosen, t = 0.19)	**30 %**	**0.45 (about 1 call in 2 is useful)**
70 %	43 %	about 0.36 (1 in 3)
80 %	more than 50 %	below 0.35

At 60 % the team catches most defaulters while calling under a third of the portfolio. The threshold is far below 0.5 because defaults are rare: the predicted probabilities sit around the 22 % base rate, so a cut-off of 0.5 flags only the most extreme cases.

6. Single test

The final pipeline (k-NN, k = 201) was refitted on all 24,000 training clients, the 0.19 threshold applied, and the 6,000 test clients scored once. It catches 61.1 % of defaulters with precision 0.436, and the test scores match cross-validation.

Metric	Cross-validation (train)	Test (used once)
Average precision	0.517	0.516
ROC-AUC	0.760	0.753
Recall at t = 0.19	≥ 0.60 (goal)	0.611
Precision at t = 0.19	0.45	0.436
F1 at t = 0.19	—	0.509

Confusion matrix (test set, threshold 0.19; rows = truth, columns = prediction):

	Flagged pays	Flagged defaulter
Actually pays (4,673)	3,625 correctly left alone	1,048 false alarms
Actually defaults (1,327)	516 missed	**811 caught**

What this means for the credit risk team:

- The tool flags **1,859 clients** (31 % of the portfolio); 811 of these calls reach a real defaulter.
- Precision 0.436 is twice the 22 % hit rate of calling clients at random.
- **516 defaulters are missed**. This is the costly error we accepted in exchange for a manageable call list.
- **1,048 clients who would have paid (22 % of them) receive a needless call**.

Generalisation. Recall on the test set (0.611) lands next to the 60 % targeted out-of-fold, so the threshold generalises. Average precision is the same in CV and on the test (0.517 vs 0.516), so there

is no sign of overfitting. The exact test ROC-AUC is 0.7535; depending on the machine it prints 0.753 or 0.754, with identical predictions.

7. Interpretation

Recent payment behaviour drives the risk: last month's delay (PAY_0) is by far the strongest signal. k-NN has no coefficients, so we read the tuned logistic regression (C = 0.1), which is statistically tied with it (section 4) and learns from the same features. Features are standardised, so each coefficient is the change in log-odds for +1 standard deviation, other features fixed.

- **PAY_0 is the strongest (+0.66).** One more standard deviation of delay last month multiplies the odds of default by e^0.66 ≈ 1.9. PAY_2 and PAY_3 are also positive but much smaller: the most recent month matters most.
- **Recent payments (PAY_AMT1, PAY_AMT2) are negative**. Clients who paid more in recent months are less risky.
- **LIMIT_BAL is negative**. Clients with a higher credit limit default less, probably because the bank already gave larger limits to clients it trusted.
- **BILL_AMT1 is clearly negative while BILL_AMT2–5 are slightly positive**. The six bills are strongly correlated (a balance carries over month to month), so the model splits their effect arbitrarily: only their combined effect is meaningful. EDUCATION_4 ("others") has a large negative coefficient, but it is a very small group, so it is weak evidence. Also, all one-hot columns are kept, so these coefficients only compare categories with each other. SEX_1 (male) is slightly positive, which links to the fairness risk in section 9.

8. Two limits

1. **The model mostly reads past delays, so it misses first-time defaulters**. PAY_0 dominates, and a client who has never been late but is about to default looks safe. The separation stays moderate (ROC-AUC 0.75): even at our threshold, 516 of 1,327 test defaulters (39 %) are missed. Data the bank does not give us, such as income or employment, would be needed to do better.
2. **One bank, one country, one period**. The data covers a single Taiwanese bank over six months of 2005. Behaviour, products and the economy have changed since, so the model may not transfer to today or to another market without retraining. LIMIT_BAL is itself a past decision of that bank, so the model partly learns the bank's earlier judgement rather than the client's real ability to pay.

9. Risks of using it on real people

The same threshold does not treat men and women alike, so the tool must stay a decision aid with a person in charge. We measured error rates per group on out-of-fold predictions (training set, threshold 0.19), so the test set was not used again.

Group (training set)	Clients	Default rate	Recall (defaulters caught)	False alarm rate (safe clients flagged)
Men	9,486	24.4 %	65.6 %	25.4 %
Women	14,514	20.6 %	57.6 %	18.8 %

Errors, fairness and privacy. About 4 in 10 defaulters are missed, and 22 % of clients who would have paid are flagged; a flag could lead to a pressing call or a lower limit for someone who did nothing wrong, so the tool should only rank clients for a human review, never act alone. The model also treats groups differently: men default more often in the data, so the model rates them riskier, and safe men

are flagged more often than safe women (25 % vs 19 %). Removing SEX would not fix this on its own, since AGE, MARRIAGE or LIMIT_BAL can carry the same information; error rates must be monitored per group, and creditworthiness scoring is classed as high-risk under the EU AI Act. Finally, the data is personal financial information: in a real deployment it falls under the GDPR (a stated purpose, secure storage, limited access) and clients have a right to an explanation of decisions that affect them, which a k-NN model ("you resemble 201 clients who defaulted") gives poorly.

10. Reproducibility and sources

Every number in this report comes from notebooks/analysis.ipynb, which imports the credit_default package instead of duplicating code. The package reproduces the same results from the command line.

- Package: DatasetLoader (data.py: load, clean, split), the ColumnTransformer (features.py), Trainer (models.py: pipelines, cross-validation, save and load with joblib), metrics and threshold (evaluate.py), and the command-line tool (cli.py).
- Configuration: config.yaml holds the paths, the seed (42), the hyperparameter grids, the final model and the threshold (0.19). No path is hard-coded.
- Run it: pip install -e ., then pytest, then python -m credit_default.cli train, evaluate and predict --input new_case.json.
- AI use: declared file by file in AI_USAGE.md.

Sources

- Yeh, I.-C. (2009). Default of Credit Card Clients [Dataset]. UCI Machine Learning Repository, CC BY 4.0. https://doi.org/10.24432/C55S3H
- Course material: Machine Learning sessions S1–S8 and Labs 1–2 (protocol: choices on the training set with cross-validation, single test).